`EmailSender` instead of creating it manually. All the `new` noise has gone, and you can focus purely on what the endpoint handler is doing: calling `EmailSender` and returning a `string` message.

Listing 8.4 Sending an email using DI to inject dependencies

```
string RegisterUser(string username, EmailSender emailSender)   ❶
{
    emailSender.SendEmail(username);                             ❷
    return $"Email sent to {username}!";                         ❷
}
```

❶ Instead of creating the dependencies implicitly, injects them directly

❷ The handler is easy to read and understand again.

One advantage of a DI container is that it has a single responsibility: creating objects or services. The minimal API infrastructure asks the DI container for an instance of a service, and the container takes care of figuring out how to create the dependency graph, based on how you configure it.

NOTE It's common to refer to services when talking about DI containers, which is slightly unfortunate, as *services* is one of the most overloaded terms in software engineering! In this context, a *service* refers to any class or interface that the DI container creates when required.

The beauty of this approach is that by using explicit dependencies, you never have to write the mess of code you saw in listing 8.3. The DI container can inspect your service's constructor and work out how to write much of the code itself. DI containers are always configurable, so if you *want* to describe how to create an instance of a service manually, you can, but by default you shouldn't need to.

TIP ASP.NET Core supports constructor injection and injection into endpoint handler methods out of the box. Technically, you can inject